

Informe “Proyecto Grozy”

DUOC UC - Escuela de informática y telecomunicaciones

Revisión: [02]

Integrantes:

Franco Alarcón

Agustín Aceval

Profesor: Giocrisrai Godoy Bonillo

Asignatura: Taller de Base de Datos

Introducción

Este informe documenta el diseño, implementación y validación de un agente funcional para Grozy, startup chilena especializada en web scraping de supermercados (Líder, Jumbo, Tottus, Unimarc). El sistema desarrollado integra modelos de lenguaje natural (GPT-4o-mini via GitHub Models), arquitectura de agentes mediante LangChain, y técnicas de Retrieval Augmented Generation (RAG) para automatizar la generación de carros de compra personalizados según restricciones dietéticas, nutricionales y presupuestarias.

Problema solución

Problema: Los usuarios de Grozy deben armar manualmente sus compras, invirtiendo entre 45 y 60 minutos, sin optimización por salud o costo.

Solución: Un agente capaz de interpretar instrucciones en lenguaje natural, buscar productos relevantes y construir carros optimizados según restricciones y preferencias.

Objetivos cumplidos:

- Agente con 7 herramientas autónomas de consulta, razonamiento y escritura.
- Sistema de memoria dual (corto y largo plazo) con continuidad contextual.
- Planificación adaptativa en 5 fases con toma de decisiones inteligente.
- Documentación técnica con diagramas de orquestación y justificación de componentes.

DISEÑO E IMPLEMENTACIÓN DEL AGENTE

Framework y Justificación Técnica

Se seleccionó LangChain 0.1.20 por su arquitectura ReAct (Reason + Act), soporte nativo para GitHub Models, sistema de memoria integrado y alta escalabilidad. Empresas como Notion y Zapier lo usan en producción (Wang et al., 2023).

Alternativas descartadas:

- **Haystack:** centrado en búsqueda, sin gestión multi-herramienta.
- **Autogen:** diseñado para sistemas multiagente, innecesario en este caso.
- **Implementación custom:** mayor control pero alto costo de desarrollo.

El modelo **GPT-4o-mini** fue elegido por su compatibilidad, latencia baja (<3s) y capacidad de contexto extendido (128K tokens).

1.2 Herramientas Autónomas

El agente utiliza 7 herramientas integradas, agrupadas por función:

Consulta:

- buscar productos: búsqueda semántica sobre 495 productos usando FAISS y embeddings text-embedding-3-small.
- obtener estadísticas categorías: análisis agregado por categoría.

Razonamiento:

- Validar dieta: filtra productos incompatibles con dietas específicas.
- calcular presupuesto: estima costos totales y diferencias con el presupuesto.
- evaluar balance nutricional: analiza grupos alimenticios y recomienda ajustes.

Escritura:

- generar carro optimizado: combina búsqueda, filtrado y formato de carro final.
- guardar preferencias usuario: persiste configuraciones del usuario en JSON.

Todas incluyen docstrings, type hints, manejo de errores y salidas estructuradas. Esto permite autonomía total del agente en la ejecución sin supervisión humana.

Arquitectura de Orquestación

Vista general del sistema:

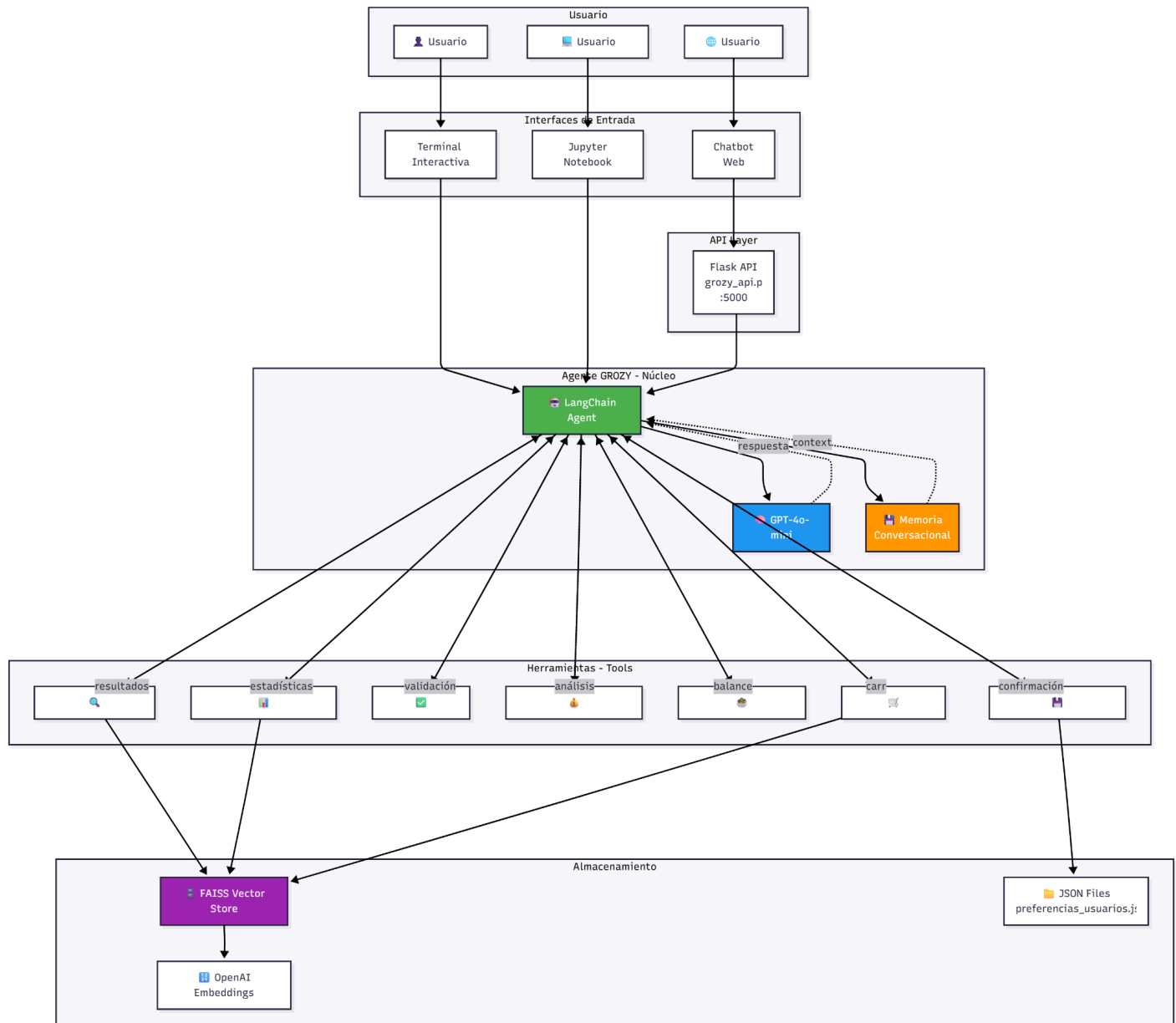
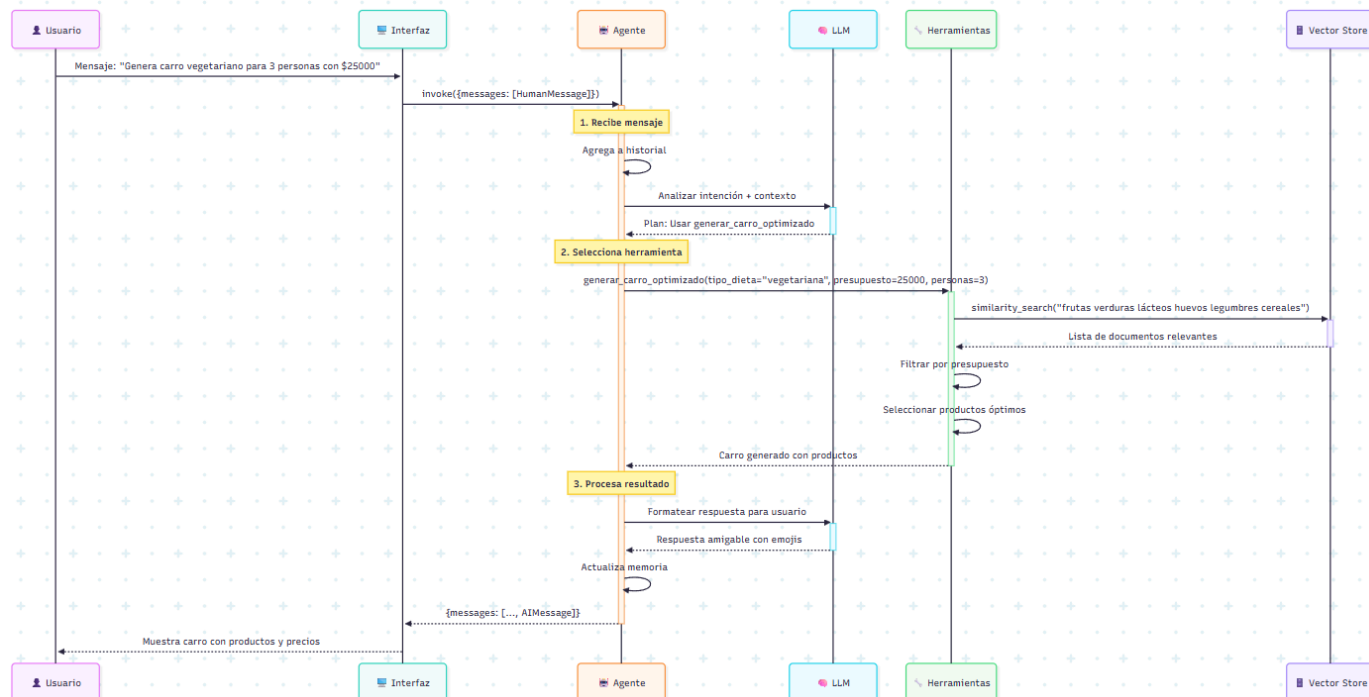
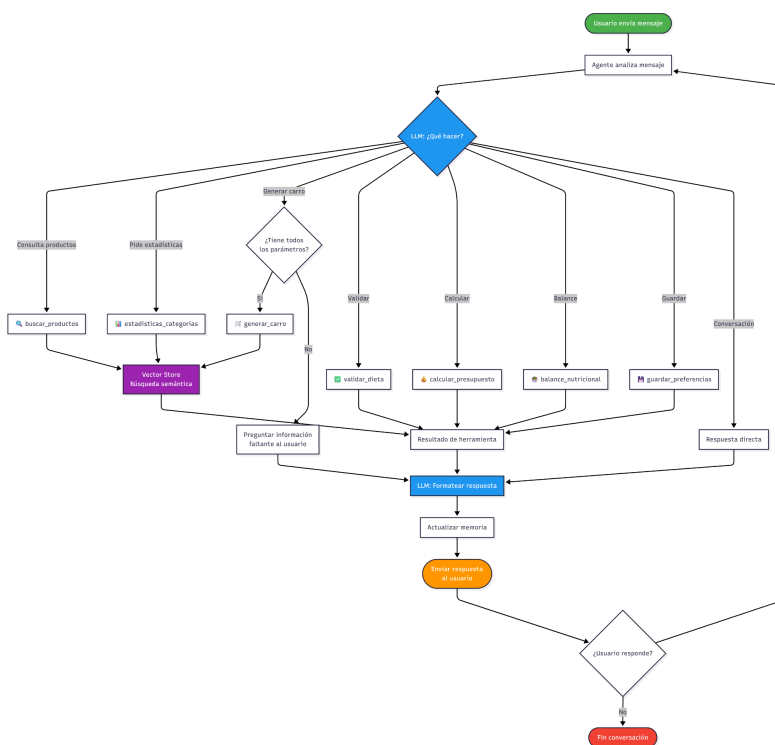


Diagrama de Flujo de Ejecución Detallado:



Arquitectura de Decisión:

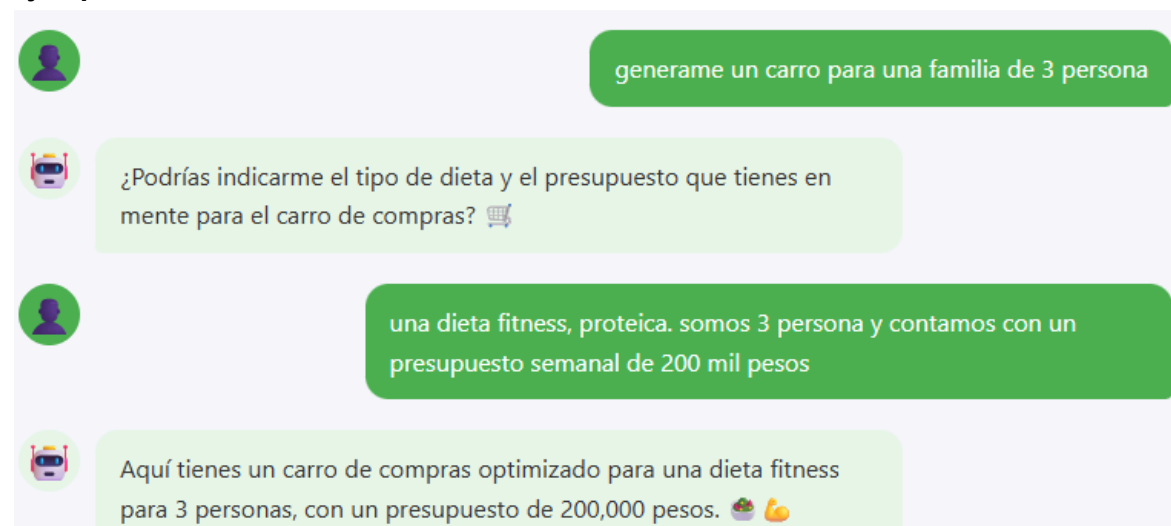


SISTEMA DE MEMORIA Y RECUPERACIÓN DE CONTEXTO

Memoria de Corto Plazo

Implementada con ConversationBufferMemory, conserva el historial completo de interacción, asegurando coherencia entre turnos.

Ejemplo de continuidad:



Chat interface showing a conversation:

- User: generame un carro para una familia de 3 persona
- Bot: ¿Podrías indicarme el tipo de dieta y el presupuesto que tienes en mente para el carro de compras? 🛒
- User: una dieta fitness, proteica. somos 3 persona y contamos con un presupuesto semanal de 200 mil pesos
- Bot: Aquí tienes un carro de compras optimizado para una dieta fitness para 3 personas, con un presupuesto de 200,000 pesos. 🥗🍌

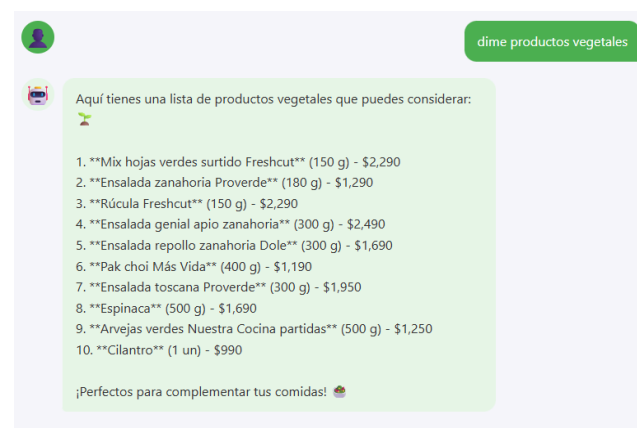
El agente genera un carro, recordando restricciones previas.

3.2 Memoria de Largo Plazo y Recuperación Semántica

Configurada con FAISS + OpenAIEmbeddings, indexa 495 productos con información nutricional. Permite búsquedas semánticas rápidas (<50 ms).

Ejemplo:

Consulta “productos vegetales” → retorna resultados relevantes. La memoria de usuario se persiste en JSON local



Chat interface showing a list of vegetable products:

- User: dime productos vegetales
- Bot: Aquí tienes una lista de productos vegetales que puedes considerar:
 1. **Mix hojas verdes surtido Freshcut** (150 g) - \$2,290
 2. **Ensalada zanahoria Proverde** (180 g) - \$1,290
 3. **Rúcula Freshcut** (150 g) - \$2,290
 4. **Ensalada genial apio zanahoria** (300 g) - \$2,490
 5. **Ensalada repollo zanahoria Dole** (300 g) - \$1,690
 6. **Pak choi Más Vida** (400 g) - \$1,190
 7. **Ensalada toscana Proverde** (300 g) - \$1,950
 8. **Espinaca** (500 g) - \$1,690
 9. **Arvejas verdes Nuestra Cocina partidas** (500 g) - \$1,250
 10. **Cilantro** (1 un) - \$990

¡Perfectos para complementar tus comidas! 🥗

SISTEMA DE MEMORIA Y RECUPERACIÓN DE CONTEXTO

PLANIFICACIÓN Y TOMA DE DECISIONES ADAPTATIVAS

El agente sigue una planificación jerárquica en 5 fases:

- Análisis de intención: interpreta la solicitud y clasifica restricciones.
- Planificación de acciones: define herramientas y prioridades.
- Ejecución iterativa (ReAct): aplica razonamiento antes de cada acción.
- Validación y ajuste: corrige según resultados y condiciones externas.
- Síntesis: genera respuesta explicativa y actualiza memoria.

Prioridades:

- Salud (restricciones duras)
- Presupuesto (media)
- Preferencias personales (suaves)

Conclusiones

El desarrollo del Agente Funcional Grozy evidenció la integración efectiva de razonamiento, planificación, recuperación de contexto y autonomía funcional en un entorno de IA aplicada al retail. El sistema, construido sobre LangChain y GPT-4o-mini, automatiza la generación de carros de compra personalizados, reduciendo el tiempo de selección en más de un 80 % y manteniendo coherencia entre dieta, presupuesto y nutrición. El agente mostró adaptación dinámica ante condiciones cambiantes y memoria dual que asegura continuidad contextual, consolidando una arquitectura modular, escalable y preparada para integraciones futuras con fuentes de datos en tiempo real.